University of British Columbia, Vancouver
Department of Computer Science

CPSC 304 Project Cover Page

Milestone #: 2

Date: February 13th, 2026

Group Number: 91

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Samuel Liao	45704046	n5a4b	samueljl321@gmail.com
Andrew Li	36020287	v1c7k	andrew.h.li2020@gmail.com
Felicia Chen	79031969	j5h9m	feliciachen200@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.
In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

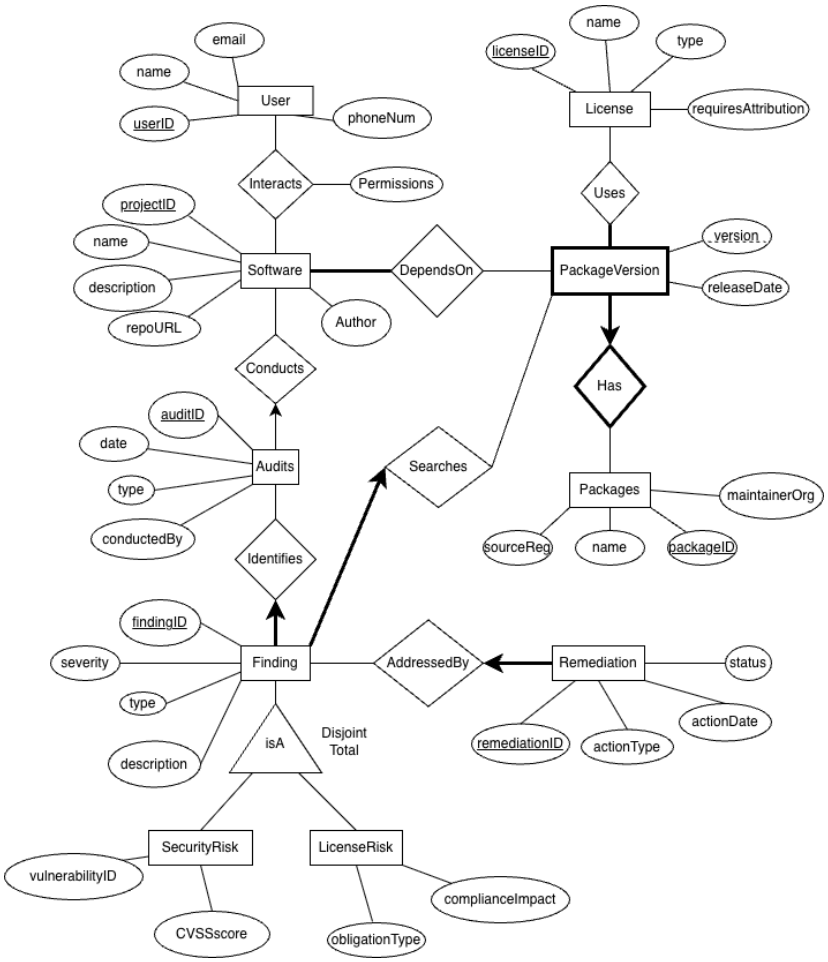
2. A brief (~2-3 sentences) summary of your project. Many of your TAs are managing multiple projects so this will help them remember details about your project.

Our Open-Source Dependency Risk & License Compliance Database aims to solve the problem of keeping track of legal obligations, security risks, and policy requirements when it comes to hundreds, if not thousands of open-source libraries (dependencies in this case), that are referenced in modern day software projects. Furthermore, offering potential solutions along the way as a form of remediation. In this case, the domain of this project would be logistics, version control, legality and software management.

3. State which database and programming language you wish to use for the project (e.g., Javascript/React and the department provided Oracle database). You will not be able to make changes to this decision after this point.

- Database of choice: Department provided Oracle database
- Programming language of choice: JavaScript/React

4. ER Diagram



5. Schema derived from ER Diagram

IMPORTANT* NAME CHANGES!

The follow things were changed to match with our TA's comments and Oracle documentation:

1. **Author** was **removed** and added as a property of Software of type VARCHAR
2. Property **version#** for **PackageVersion, Finding, DependsOn** and **User**, was changed to **version** as the special character '#' is not recommended in oracle naming convention
3. Property **phone#** of User was changed to **phoneNum** following same rationale as above
4. **Package** was changed to **Packages** to prevent overriding special meaning of Oracle keywords in accordance with this.
5. **Audit** was changed to **Audits** as it is a reserved word in Oracle according to the same documentation above
6. Property **permissions** was moved from **User** to **Interacts**, due to the conditional aspect of user permissions when interacting with different software

User(userID: INTEGER, name: VARCHAR, email: VARCHAR, phoneNum: VARCHAR)

PK: userID
CK: userID, {email, phoneNum}
FK: none
Constraints: email NOT NULL UNIQUE, phoneNum NOT NULL, name NOT NULL

Interacts(userID: INTEGER, projectID: INTEGER, permissions: VARCHAR)

PK: {userID, projectID}
CK: {userID, projectID}
FK: userID → User(userID), projectID → Software(projectID)
Constraints: permissions NOT NULL

Software(projectID: INTEGER, name: VARCHAR, description: VARCHAR, repoURL: VARCHAR, author: VARCHAR)

PK: projectID
CK: projectID, repoURL
FK: none
Constraints: name NOT NULL, repoURL NOT NULL UNIQUE, author NOT NULL

Audits(auditID: INTEGER, projectID: INTEGER, date: DATE, type: VARCHAR, conductedBy: VARCHAR)

PK: auditID
CK: auditID
FK: projectID → Software(projectID)
Constraints: date NOT NULL, type NOT NULL, conductedBy NOT NULL

Finding(findingID: INTEGER, auditID: INTEGER, severity: INTEGER, type: VARCHAR, description: VARCHAR, packageID: INTEGER, version: INTEGER)
PK: findingID
CK: findingID
FK: auditID → Audits(auditID),
{packageID, version} → PackageVersion(packageID, version)
Constraints: severity NOT NULL, type NOT NULL, description NOT NULL

SecurityRisk(findingID: INTEGER, vulnerabilityID: INTEGER, CVSSscore: DECIMAL(3, 1))
PK: findingID
CK: findingID
FK: findingID → Finding(findingID)
Constraints: vulnerabilityID NOT NULL, CVSSscore NOT NULL

LicenseRisk(findingID: INTEGER, obligationType: INTEGER, complianceImpact: VARCHAR)
PK: findingID
CK: findingID
FK: findingID → Finding(findingID)
Constraints: obligationType NOT NULL, complianceImpact NOT NULL

Remediation(remediationID: INTEGER, findingID: INTEGER, actionType: VARCHAR, actionDate: DATE, status: INTEGER)
PK: remediationID
CK: remediationID
FK: findingID → Finding(findingID)
Constraints: actionType NOT NULL, actionDate NOT NULL, status NOT NULL

PackageVersion(packageID: INTEGER, version: INTEGER, releaseDate: DATE)
PK: {packageID, version}
CK: {packageID, version}
FK: packageID → Packages(packageID)
Constraints: releaseDate NOT NULL

Packages(packageID: INTEGER, name: VARCHAR, sourceReg: VARCHAR, maintainerOrg: VARCHAR)

PK: packageID
CK: packageID
FK: none
Constraints: name NOT NULL, sourceReg NOT NULL, maintainerOrg NOT NULL

DependsOn(projectID: INTEGER, version: INTEGER, packageID: INTEGER)

PK: {projectID, version, packageID}
CK: {projectID, version, packageID}
FK: projectID → Software(projectID),
{version, packageID} → PackageVersion(packageID, version)
Constraints: none

License(licenseID: INTEGER, name: VARCHAR, type: VARCHAR, requiresAttribution: BOOLEAN)

PK: licenseID
CK: licenseID
FK: none
Constraints: name NOT NULL, type NOT NULL, requiresAttribution NOT NULL

Uses(licenseID: INTEGER, version: INTEGER, packageID: INTEGER)

PK: {licenseID, version, packageID}
CK: {licenseID, version, packageID}
FK: licenseID → License(licenseID),
{packageID, version} → PackageVersion(packageID, version)
Constraints: none

6. Functional Dependencies

User:

- userID → name, email, phoneNum
- email, phoneNum → userID

Interacts:

- userID, projectID → permissions

Software:

- projectID → name, description, repoURL, author
- repoURL → projectID

Audits:

- projectID, date → type, conductedBy

Finding:

- auditID, packageID, version, type → severity

SecurityRisk:

- findingID → vulnerabilityID, CVSSscore

LicenseRisk:

- findingID → obligationType, complianceImpact

Remediation:

- remediationID → actionType, actionDate, status, findingID

PackageVersion:

- packageID, version → releaseDate

Packages:

- packageID → name, sourceReg, maintainerOrg

DependsOn:

- No non-trivial FDs

License:

- licenseID → name, type, requiresAttribution

Uses:

- No non-trivial FDs

7. Normalization

We use the lossless-join BCNF decomposition algorithm in order to ensure that the relations follow the BCNF specification. Since all relations are in BCNF except for the Audits and

Finding relations, we simply need to apply the algorithm to these relations. All other relations stay the same.

To begin we look at the Audits relation and schema to identify the problem:

Schema: Audits(auditID: INTEGER PK, projectID: INTEGER FK, date: DATE, type: VARCHAR, conductedBy: VARCHAR)

Given FD: projectID, date → type, conductedBy

Primary key (from schema): auditID

We have a non-trivial FD:

projectID, date → type, conductedBy

Since (projectID, date) is not the primary key, the determinant is not a superkey.

1. This **violates BCNF**
2. This also **violates 3NF** (since RHS attributes are non-prime)

Therefore, we decompose using the violating FD into:

AuditEvent(projectID: INTEGER FK, date: DATE, type: VARCHAR, conductedBy: VARCHAR)

Primary key: (projectID, date)

Determinant is the key thus satisfying BCNF

Now we have:

Audits(auditID: INTEGER PK, projectID: INTEGER FK, date: DATE)

Primary key: auditID

Non non-trivial FDs remain

Giving the final BCNF Decomposition of:

AuditEvent(projectID: INTEGER, date: DATE, type: VARCHAR, conductedBy: VARCHAR)

PK: {projectID, date}

CK: {projectID, date}

FK: projectID → Software(projectID)

Constraints: type NOT NULL, conductedBy NOT NULL

Audits(auditID: INTEGER PK, projectID: INTEGER FK, date: DATE)

PK: auditID

CK: auditID

FK: projectID → Software(projectID)

Constraints: date NOT NULL

For Finding:

Schema: Finding(findingID: INTEGER PK, auditID: INTEGER FK, severity: INTEGER, type: VARCHAR, description: VARCHAR, packageID: INTEGER FK, version: INTEGER FK)

Given FD: auditID, packageID, version, type → severity

Primary key (from schema): findingID

We have a non-trivial FD:

auditID, packageID, version, type → severity

Similarly, (auditID, packageID, version, type) is not a superkey, violating both 3NF and BCNF.

$$\{auditID,\ packageID,\ version,\ type\}^{+} = \{auditID,\ packageID,\ version,\ type,\ severity\}$$

Therefore, we decompose into:

FindingSeverity(auditID: INTEGER, packageID: INTEGER, version: INTEGER, type: VARCHAR, severity: INTEGER)

PK: {auditID, packageID, version, type}
CK: {auditID, packageID, version, type}
FK: auditID → Audits(auditID),
{packageID, version} → PackageVersion(packageID, version)
Constraints: severity NOT NULL

Finding(findingID: INTEGER, auditID: INTEGER, type: VARCHAR, description: VARCHAR, packageID: INTEGER, version: INTEGER)

PK: findingID
CK: findingID
FK: auditID → Audits(auditID),
{packageID, version} → PackageVersion(packageID, version)
Constraints: type NOT NULL, description NOT NULL

Now, FindingSeverity satisfies the FD: auditID, packageID, version, type → severity.

8. SQL DDL Statements

```
CREATE TABLE Software (
    projectID       INTEGER PRIMARY KEY,
    name            VARCHAR(1024) NOT NULL,
    description     VARCHAR(1024),
    repoURL         VARCHAR(1024) NOT NULL UNIQUE,
    author          VARCHAR(1024) NOT NULL
)

CREATE TABLE Audits (
    auditID         INTEGER PRIMARY KEY,
    projectID       INTEGER,
    date            DATE NOT NULL,
    FOREIGN KEY (projectID) REFERENCES Software(projectID)
        ON DELETE CASCADE
```

```
                ON UPDATE CASCADE
)

CREATE TABLE AuditEvent (
        projectID               INTEGER,
        date                    DATE NOT NULL,
        type                    VARCHAR(1024) NOT NULL,
        conductedBy             VARCHAR(1024) NOT NULL,
        PRIMARY KEY (projectID, date),
        FOREIGN KEY (projectID) REFERENCES Software(projectID)
                ON DELETE CASCADE
                ON UPDATE CASCADE
)

CREATE TABLE Finding (
        findingID               INTEGER PRIMARY KEY,
        auditID                 INTEGER,
        type                    VARCHAR(1024) NOT NULL,
        description             VARCHAR(1024) NOT NULL,
        packageID               INTEGER,
        version                 INTEGER,
        FOREIGN KEY (auditID) REFERENCES Audits(auditID)
                ON DELETE CASCADE
                ON UPDATE CASCADE
        FOREIGN KEY (packageID, version) REFERENCES PackageVersion(packageID,
        version) ON DELETE CASCADE
)

CREATE TABLE FindingSeverity (
        auditID                 INTEGER,
        packageID               INTEGER,
        version                 INTEGER,
        severity                INTEGER NOT NULL,
        type                    VARCHAR(1024) NOT NULL,
        PRIMARY KEY (auditID, packageID, version, type),
        FOREIGN KEY (auditID) REFERENCES Audits(auditID)
                ON DELETE CASCADE
                ON UPDATE CASCADE
        FOREIGN KEY (packageID, version) REFERENCES PackageVersion(packageID,
        version) ON DELETE CASCADE
```

```
)

CREATE TABLE SecurityRisk (
	findingID		INTEGER PRIMARY KEY,
	vulnerabilityID		INTEGER NOT NULL,
	CVSSscore		DECIMAL(3, 1) NOT NULL,
	FOREIGN KEY (findingID) REFERENCES Finding(findingID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
)

CREATE TABLE LicenseRisk (
	findingID		INTEGER PRIMARY KEY,
	obligationType		INTEGER NOT NULL,
	complianceImpact	VARCHAR(1024) NOT NULL,
	FOREIGN KEY (findingID) REFERENCES Finding(findingID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
)

CREATE TABLE Remediation (
	remediationID		INTEGER PRIMARY KEY,
	findingID		INTEGER,
	actionType		VARCHAR(1024) NOT NULL,
	actionDate		DATE NOT NULL,
	status			INTEGER NOT NULL,
	FOREIGN KEY (findingID) REFERENCES Finding(findingID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
)

CREATE TABLE PackageVersion (
	packageID		INTEGER,
	version			INTEGER,
	releaseDate		DATE NOT NULL,
	PRIMARY KEY (packageID, version),
	FOREIGN KEY (packageID) REFERENCES Packages(packageID)
		ON DELETE CASCADE
)
```

```
CREATE TABLE Packages (
	packageID		INTEGER PRIMARY KEY,
	name			VARCHAR(1024) NOT NULL,
	sourceReg		VARCHAR(1024) NOT NULL,
	maintainerOrg		VARCHAR(1024) NOT NULL
)

CREATE TABLE DependsOn (
	projectID		INTEGER,
	version			INTEGER,
	packageID		INTEGER,
	PRIMARY KEY(projectID, version, packageID),
	FOREIGN KEY(projectID) REFERENCES Software(projectID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
	FOREIGN KEY (version, packageID) REFERENCES PackageVersion(version,
	packageID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
)

CREATE TABLE License (
	licenseID		INTEGER PRIMARY KEY,
	name			VARCHAR(1024) NOT NULL,
	type			VARCHAR(1024) NOT NULL,
	requiresAttribution	BOOLEAN NOT NULL
)

CREATE TABLE Uses (
	licenseID		INTEGER,
	version			INTEGER,
	packageID		INTEGER,
	PRIMARY KEY (licenseID, version, packageID),
	FOREIGN KEY (licenseID) REFERENCES License(licenseID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
	FOREIGN KEY (version, packageID) REFERENCES PackageVersion(version,
	packageID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
```

```
)

CREATE TABLE User (
	userID		INTEGER PRIMARY KEY,
	name		VARCHAR(1024) NOT NULL,
	email		VARCHAR(1024) NOT NULL UNIQUE,
	phoneNum	VARCHAR(1024) NOT NULL
)

CREATE TABLE Interacts (
	userID		INTEGER,
	projectID	INTEGER,
	permissions	VARCHAR(1024) NOT NULL,
	PRIMARY KEY (userID, projectID),
	FOREIGN KEY (userID) REFERENCES User(userID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
	FOREIGN KEY (projectID) REFERENCES Software(projectID)
		ON DELETE CASCADE
		ON UPDATE CASCADE
)
```

Context for ON DELETE and ON UPDATE choices:

- We chose to do ON DELETE CASCADE because all the tables that involve this operation are either relationships between entities, or ISA subclasses. Therefore, when we delete either the entities or the superclass, it would not make sense to keep a reference to the relationship/subclass.
- We were told by a **TA** that ON UPDATE CASCADE is now supported by Oracle, so in that case we would want relationships/subclasses to be updated appropriately when the entities/superclass are updated. That being said, the **lecture slides** did mention that Oracle does NOT support ON UPDATE CASCADE, so if that is the case, we would seek an Oracle-compatible alternative.

9. An explicit acknowledgment about your use of AI tools in this assignment.

No AI tools were used in the creation of this milestone.